

FIAP GRADUAÇÃO

SISTEMA DE INFORMAÇÃO

OBJECT-ORIENTED PROGRAMMING AND JAVA WEB DEVELOPMENT

PROF. LUCAS GONZALEZ

- Conceitos de Orientação a Objetos e implementação
 - Polimorfismo
 - Sobrecarga
 - Modificador Final
 - Abstract
 - Interface
 - Anônima

POLIMORFISMO - SOBREPOSIÇÃO

- É a implementação de métodos em subclasses de tal forma que **anule** o comportamento que ele apresentava em sua superclasse ou apenas **acrescente** novas instruções;
- Para ocorrer a sobreposição é necessário que o método tenha a **mesma assinatura** do método da superclasse;
- **Variáveis de instância** não são sobrepostas porque não definem nenhum comportamento;
- Métodos definidos como **final** não podem ser sobrepostos;

SOBREPOSIÇÃO X SOBRECARGA

```
public class Teste {  
    public void imprimir() {  
        System.out.println("Teste!!");  
    }  
}
```

```
public class Novidade extends Teste {  
    public void imprimir() {  
        System.out.println("Novidade!!");  
    }  
    public void imprimir(String msg) {  
        System.out.println(msg);  
    }  
}
```



SOBREPOSIÇÃO



SOBRECARGA

SOBRECARGA

- A sobrecarga não tem relação com a **herança** e o **polimorfismo**;
- Implementação em **uma mesma classe** de duas ou mais versões para um mesmo método, de forma que manifestem comportamentos diferentes;
- Não é possível implementar o mesmo método duas vezes com os mesmos tipos de parâmetros;
- Sobrecarregar um método significa implementar duas ou mais versões do método com tipos de **parâmetros distintos**;
- Métodos sobrecarregados podem ter retornos diferentes, mas alterar apenas o tipo de retorno não configura a sobrecarga;

```
public int quadrado(int x, int y){  
    return Math.pow(x, y);  
}
```

```
public double quadrado(double x, double y){  
    return Math.pow(x, y);  
}
```

I MODIFICADOR FINAL

- O modificador **final** pode ser usado para **classes**, **variáveis** e **métodos**, tendo um comportamento diferente para cada um;
- Uma **classe** definida com o modificador final **não pode ser estendida**, ou seja, não podemos aplicar herança na classe;
- O modificar final aplicado a uma **variável** indica que a mesma é uma **constante**, ou seja, tem um valor inicial que não pode ser alterado durante a execução do programa;
- Para **métodos**, o modificar final indica que o método **não pode ser sobrescrito**, ou seja, não pode ser redefinido nas subclasses;

CLASSES ABSTRATAS

- São classes que **não podem ser instanciadas** e tem o propósito único de atuar como uma **superclasse**;
- Quando usar uma classe abstrata?
 - Quando a classe apresentar pelo menos um **método abstrato**.
 - Exemplo: superclasse Forma e subclasses Círculo e Cilindro.
 - Qual o sentido da existência de um objeto Forma? Quais seriam seus atributos? Qual seria seu comportamento?
 - Nesse caso, a classe Forma foi definida como abstrata;
- Uma classe que não é abstrata é chamada de **concreta**;
- Ao projetar uma hierarquia de herança você terá de decidir quais classes serão abstratas e quais serão concretas;

I MÉTODOS ABSTRATOS

- Uma classe abstrata deve ser estendida;
- Um método abstrato significa que ele **deve** ser sobreposto;
- Em algumas situações, você pode chegar a conclusão que alguns métodos não tem sentido ter código na superclasse. Tais métodos devem ser definidos como abstratos;
- Um método abstrato **não tem corpo!!**
- Exemplo:
 - **public abstract double calcular(double x, double y);**
- Caso você declare um **método** como **abstrato** deverá declarar a **classe** como **abstrata** também;

CLASSE OBJECT

- É a mãe de todas as classes, é a **superclasse** de tudo!
- **Toda classe** que você criar estenderá Object, sem que seja preciso declará-lo;
- Qualquer classe que não estender explicitamente outra classe estenderá implicitamente Object;
- Métodos da classe Object:
 - equals(Object o);
 - getClass();
 - hashCode();
 - toString();

I HERANÇA MÚLTIPLA

- Herança múltipla significa uma classe possuir **mais de uma superclasse**;
- Problemas: **conflitos** de nomes de métodos em linguagens como C++;
- **Não existe** herança múltipla no Java;
- O problema da herança múltipla é resolvido com o conceito de **interface**;

INTERFACE

- Interfaces como as classes, compõem-se de atributos e métodos, **não apresentam construtores e não podem ser instanciadas**;
- Uma interface Java é como uma classe 100% abstrata;
- Todos os atributos e métodos de uma interface devem ser **públicos**;
- Todo **atributo** em uma interface é implicitamente **final** e **estático**;
- Todo **método** em uma interface é **abstrato**;
- Métodos de interfaces **nunca podem ser estáticos** porque métodos estáticos não podem ser abstratos;

INTERFACE

```
public interface Pagamento {  
    boolean efetuarPagamento(double valor);  
}  
  
public class PagamentoCartaoCredito implements Pagamento {  
    public boolean efetuarPagamento(double valor) {  
        return valor < 100;  
    }  
}  
  
public class PagamentoPayPal implements Pagamento {  
    public boolean efetuarPagamento(double valor) {  
        return valor <= 500;  
    }  
}
```

■ CLASSE ANÔNIMA

- É uma classe sem nome;
- Criada e usada no mesmo instante;
- É utilizada para sobrescrever métodos sem criar um novo arquivo de classe;

CLASSE ANÔNIMA

```
class Animal {  
    public void makeSound() {  
        System.out.println("Animal sound");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal myAnimal = new Animal() {  
            public void makeSound() {  
                System.out.println("Woof woof");  
            }  
        }; // semicolon is required to end the line of code that creates the object  
        myAnimal.makeSound();  
    }  
}
```

CLASSE ANÔNIMA

```
interface Greeting {  
    void sayHello();  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Greeting greet = new Greeting() {  
            public void sayHello() {  
                System.out.println("Hello, World!");  
            }  
        };  
  
        greet.sayHello();  
    }  
}
```

INTERFACE

- Interfaces famosas:
 - Comparable
 - Collection
 - List
 - Set

COMPARABLE

- Permite comparação entre objetos das classes que implementam a interface;
- Utilizada para ordenação;
- Possui o método: **int compareTo(Object o);**
 - Como funciona o retorno:
 - Retorna **0** se os objetos são iguais.
 - Retorna um **valor negativo** se o **objeto atual** (this) é **menor** que o objeto passado como parâmetro.
 - Retorna um **valor positivo** se o objeto atual (this) é **maior** que o objeto passado como parâmetro.

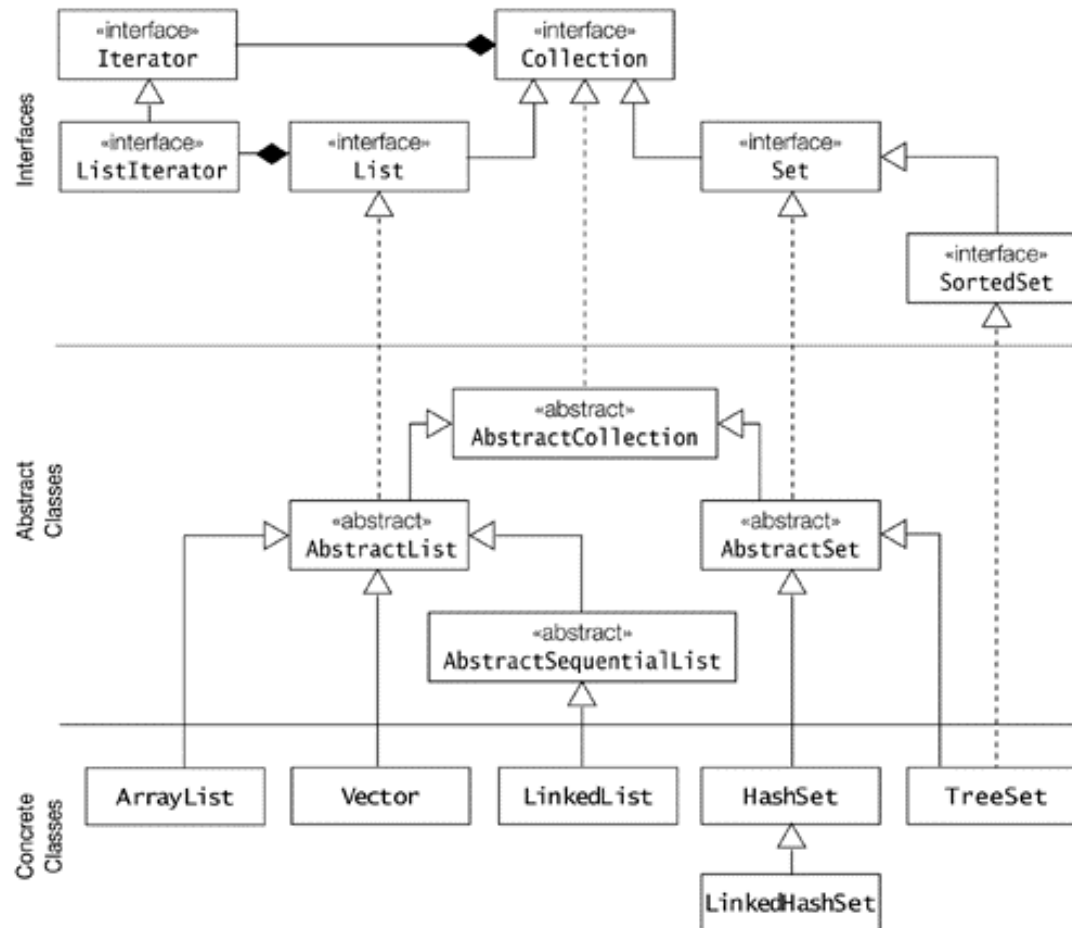
COMPARABLE

```
public class Empregado implements Comparable {  
    private String nome;  
    public String getNome(){ return nome; }  
  
    public int compareTo(Object o){  
        return this.nome.compareTo(((Empregado) obj).getNome());  
    }  
}
```

COLLECTION, LIST e SET

- São interfaces;
- Possuem os métodos:
 - **boolean contains(Object o);**
 - **boolean remove(Object o);**
 - **boolean add(Object o);**
 - **int size();**

COLLECTION, LIST e SET



<https://techtractall.wordpress.com/2017/02/10/collection-in-java/>

LIST

- Permite criar listas de dados;
- Tem sequência definida e permite duplicidade nos dados;
- O índice do primeiro elemento é zero (0);
- É utilizada através de ArrayList, Vector e LinkedList;
 - ArrayList pesquisa rápida mas não sincronizada;
 - Vector pesquisa rápida e sincronizado;
 - LinkedList inserção e remoção rápida, não sincronizada.

BIBLIOGRAFIA

- DEITEL, H. M., DEITEL, P. J. **JAVA como programar**. São Paulo: 8ª edição, Pearson Prentice Hall, 2010.

Próxima aula estudaremos

- ❑ Interfaces Set, Map e Classe Collections.





Copyright © 2025 Prof. Lucas Gonzalez

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).